

Integración de Servicios Cloud GCP

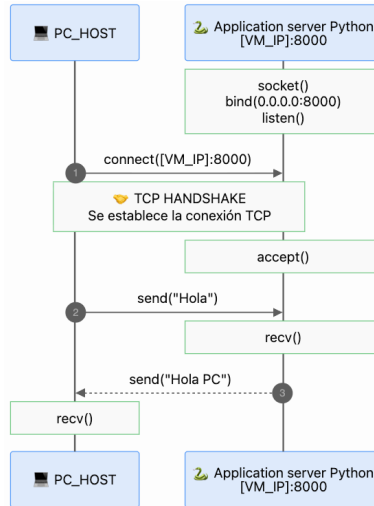
Servidor web y servidor de aplicaciones; Windows Subsystem for Linux

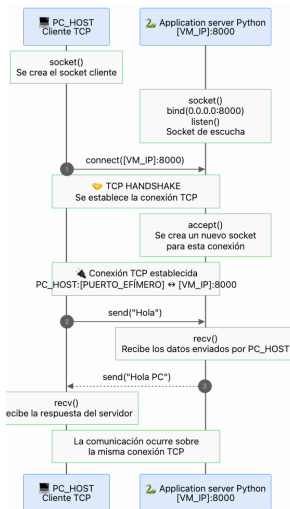
Semana 05 - Sesión 10

Rel Guzman

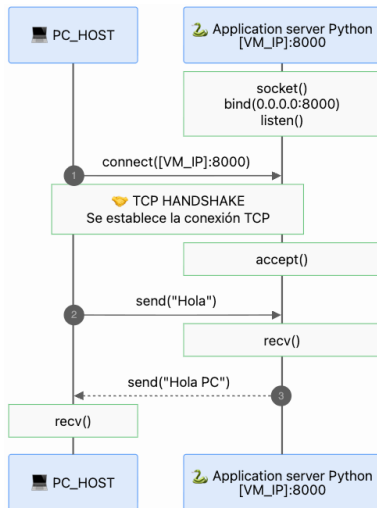


**Universidad
Tecnológica
del Perú**

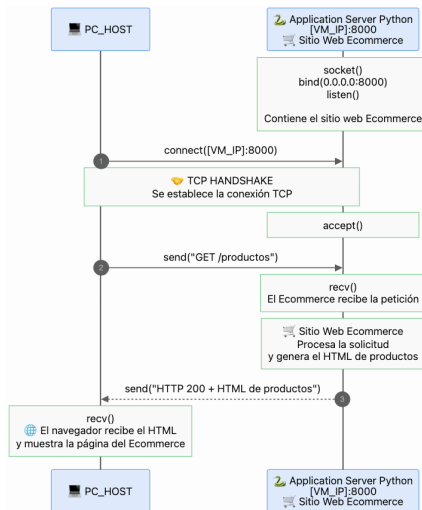




El mismo intercambio, descrito paso a paso: un socket de escucha, un socket nuevo por conexión.



Visto en resumen: `connect`, `accept` y luego `send/rcv` sobre la misma conexión.



La misma mecánica al servicio de un sitio web: recibe GET /productos y devuelve HTML.

Logro de la sesión

Al finalizar, podrás distinguir un **servidor web** de un **servidor de aplicaciones**, ubicar a `nginx` y al programa de Python de la Sesión 09 en ese esquema, explicar qué es el **Windows Subsystem for Linux (WSL)** y comparar WSL con virtualizar mediante un hipervisor.

- ▶ Cuando abres una página, ¿el servidor te envía un archivo ya guardado o arma la respuesta en ese momento?
- ▶ ¿Recuerdas qué era un hipervisor y una máquina virtual de la Sesión 02?
- ▶ Si trabajas en Windows, ¿cómo has ejecutado hasta ahora comandos de Linux?

¿Para qué te servirá lo aprendido?

Toda aplicación en producción combina un servidor web y un servidor de aplicaciones. Saber qué hace cada uno permite ubicar dónde está una falla y por qué se separan. Y WSL te da un entorno Linux dentro de Windows para practicar todo el curso sin cambiar de equipo.

Es la misma idea de siempre: cada componente con una única responsabilidad clara.

Servidor web y servidor de aplicaciones

El recorrido que montamos

```
Navegador -> nginx :80 -> sitio.py 127.0.0.1:3000  
Navegador <- nginx <----- respuesta generada por Python
```

nginx

Escuchó en el puerto público 80, recibió la solicitud HTTP y la reenvió hacia adentro (proxy inverso).

sitio.py

Ejecutó código Python en 127.0.0.1:3000 y devolvió el HTML que nginx entregó al navegador.

La pregunta de hoy

¿nginx y sitio.py hacen el mismo trabajo?

Idea central

Es un programa que **espera solicitudes HTTP** y devuelve una respuesta. En su forma más básica, esa respuesta es un archivo que ya está guardado en el disco: una página `.html`, una hoja de estilos, una imagen, un archivo de JavaScript. El servidor web solo lo *lee y lo entrega*.

También se encarga de

Aceptar muchas conexiones a la vez, atender HTTPS, registrar los accesos y aplicar límites. Es la **puerta de entrada** pública.

Ejemplos

nginx, Apache HTTP Server. En el curso usamos nginx.

Contenido estático

Se llama **estático** porque el archivo es el mismo para todos los visitantes hasta que alguien lo cambia a mano.

Idea central

Es un programa que **ejecuta el código de una aplicación** para construir la respuesta en el momento de cada solicitud. No lee un archivo terminado: corre un programa que consulta datos, aplica reglas y *genera* el resultado (una página HTML, o datos en formato JSON).

Contenido dinámico

La respuesta puede cambiar según quién pregunta, la hora o lo que haya en la base de datos. Se arma cada vez.

En la Sesión 09

`sitio.py` hizo este papel: ejecutó Python y armó el HTML al vuelo. Usaba `http.server`, que sirve para aprender el flujo, no para producción.

Ejemplos

En Python: Gunicorn o uWSGI ejecutando una aplicación hecha con un framework (Django, Flask, FastAPI).

	Servidor web	Servidor de aplicaciones
Qué devuelve	Un archivo ya guardado	Una respuesta generada en el momento
Qué hace con él	Lo lee y lo entrega	Ejecuta código que lo produce
Tipo de contenido	Estático	Dinámico
Rol habitual	Puerta de entrada, HTTPS, estáticos	Lógica de la aplicación y datos
Ejemplo	nginx, Apache	Gunicorn, uWSGI

Matiz honesto

La frontera no es un muro: un servidor web puede ejecutar algo de lógica y un servidor de aplicaciones puede entregar archivos. Aun así, los roles son distintos y por eso se combinan.

nginx = servidor web

Punto de entrada público en el puerto 80/443.
Entrega archivos estáticos, gestiona HTTPS y,
con `proxy_pass`, reenvía lo demás hacia
adentro.

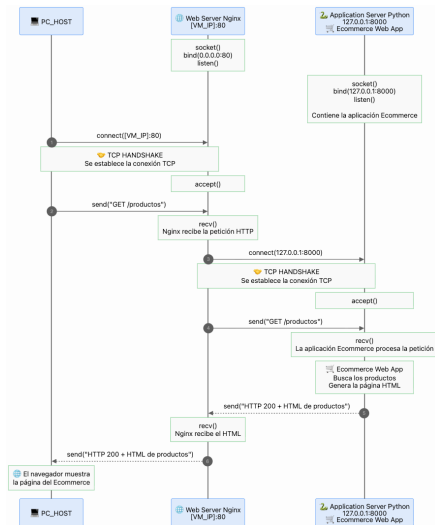
sitio.py = servidor de aplicaciones

Escuchó solo en `127.0.0.1:3000`, ejecutó la
lógica en Python y generó la respuesta
dinámica.

Precisión para producción

Una aplicación real no usa `http.server`. Usa un servidor de aplicaciones especializado (Gunicorn, uWSGI) detrás de `nginx`. El esquema es el mismo; cambia solo la pieza de atrás.

Diagrama de bloques: web + aplicación



nginx (:80) recibe la petición y la reenvía a la aplicación Python interna (:8000), que genera el HTML.

¿Por qué separar las dos piezas?

Cada una en lo suyo

El servidor web está optimizado para muchas conexiones, HTTPS y archivos. El servidor de aplicaciones se concentra en la lógica del negocio.

Seguridad

El puerto de la aplicación queda interno (127.0.0.1). Solo `nginx` se expone. Es lo que comprobaste en la Sesión 09.

Escalar por separado

Puedes poner varias copias de la aplicación detrás del mismo `nginx` y repartir la carga entre ellas.

Mantenimiento

Actualizas o reinicias la aplicación sin tocar la puerta de entrada, y al revés.

Windows Subsystem for Linux

Ejecutar Linux dentro de Windows

Punto de partida: Windows y Linux no se entienden solos

Recordemos dos términos

Sistema operativo: el software base que administra el equipo y atiende a los programas. **Kernel:** su parte central, la que habla directamente con el hardware y responde a lo que los programas piden.

Llamada al sistema

Cuando un programa necesita algo del sistema (abrir un archivo, usar la red, reservar memoria) se lo **pide al kernel**. Esa petición se llama *llamada al sistema*.

El problema

Windows y Linux tienen kernels distintos y piden esas cosas de forma diferente. Un programa hecho para Linux no corre tal cual en Windows.

Arranque dual (*dual boot*)

Instalas Windows y Linux en el mismo equipo y al encender eliges cuál usar. Solo uno funciona a la vez; para cambiar, reinicias.

Máquina virtual con hipervisor

Hipervisor tipo 2 (Sesión 02): un programa sobre Windows que crea una **máquina virtual**, una computadora simulada por software. Dentro instalas Linux, con recursos asignados y su propio escritorio.

El costo de la máquina virtual

Reservas memoria y núcleos por adelantado, arranca como un equipo aparte, vive aislada de Windows y compartir archivos entre ambos cuesta configurarlo.

Definición

Windows Subsystem for Linux es una función de Windows que te permite ejecutar un entorno GNU/Linux (la terminal, sus herramientas, paquetes que instalas con apt) **dentro de Windows**, sin reiniciar y sin que tú tengas que crear una máquina virtual.

Cómo se ve

Después de instalarlo, abres una terminal de Ubuntu como una aplicación más. Los archivos de Windows y los de Linux se ven entre sí.

Cómo se instala

Un solo comando: `wsl --install`. Windows descarga y prepara todo lo necesario.

WSL 1: una capa de traducción

Cuando el programa de Linux hace una llamada al sistema, WSL 1 la **traduce en el acto** a la llamada equivalente de Windows. No hay un kernel de Linux: hay un intérprete en el medio.

WSL 2: un kernel de Linux real

Trae un **kernel de Linux de verdad**, ejecutándose en una **máquina virtual ligera y optimizada** que gestiona la tecnología de hipervisor de Windows (Hyper-V). Es la opción por defecto y la recomendada.

Por qué se pasó a WSL 2

Traducir cada llamada tenía límites: programas que no funcionaban y accesos a disco lentos. Con un kernel real, la compatibilidad es completa y el rendimiento, mayor.

Pregunta para discutir

WSL 2 usa un hipervisor por debajo. Entonces, ¿es igual que crear una máquina virtual con VirtualBox?

	Máquina virtual tradicional	WSL 2
Arranque	La enciendes; tarda de segundos a minutos	Casi instantáneo, al abrir la terminal
Recursos Integración	Los fijas por adelantado Aislada; archivos y red aparte	Se ajustan solos según el uso Comparte archivos y red con Windows
Interfaz	Escritorio Linux completo	Una terminal (apps gráficas, opcional)

La respuesta: comparten base, pero no se usan igual

Lo que comparten

WSL 2 se apoya en la **misma tecnología de hipervisor** de Windows que usaría una máquina virtual. En ese sentido, sí hay virtualización debajo.

Lo que los diferencia

La máquina virtual está **aislada** y con recursos fijos. WSL 2 es **ligero e integrado**: se siente como parte de Windows, no como un equipo aparte.

Y WSL 1

WSL 1 no usa ningún hipervisor: es solo traducción de llamadas al sistema. Ahí la respuesta es un no rotundo.

Conclusión

WSL 2 *usa* virtualización, pero está tan integrado y es tan liviano que en la práctica lo tratas como una terminal más de Windows.

Desarrollar con herramientas de Linux

bash, apt, git, python3, ssh, gcloud y docker desde Windows, sin dejar de usar tus programas de siempre.

Probar antes de subir al servidor

Corres el script en WSL con el mismo Ubuntu que la VM de GCP; si funciona ahí, se comporta casi igual en la nube.

Aprender la CLI de Linux

Practicar permisos, servicios y nginx sin instalar una máquina virtual completa.

Qué no es

WSL es tu **entorno de trabajo local**. No reemplaza a un servidor de producción, que corre en la nube sobre un hipervisor tipo 1.

Qué necesitas

Windows 10 versión 2004 o superior (compilación 19041+), o Windows 11.

La virtualización habilitada en el firmware (BIOS/UEFI); en la mayoría de equipos ya lo está.

Comprobar si ya está

```
PS> wsl --version  
PS> wsl --status
```

Muestran la versión de WSL, la del kernel y la distribución por defecto. Si el comando no existe, aún no está instalado.

1. Abre una terminal como administrador

PowerShell o Símbolo del sistema, con clic derecho → *Ejecutar como administrador*.

2. Instala WSL

```
PS> wsl --install
```

Activa lo necesario, fija **WSL 2** por defecto y descarga **Ubuntu**.

3. Reinicia el equipo

Al volver, se abre Ubuntu automáticamente y termina de configurarse.

4. Crea tu usuario de Linux

```
Enter new UNIX username: rgap  
New password:  
Retype new password:
```

Es una cuenta de Linux, distinta de la de Windows. Este usuario será administrador vía sudo.

Desde Windows (PowerShell)

```
PS> wsl -l -v
    NAME STATE VERSION
* Ubuntu Running 2
PS> wsl --list --online
PS> wsl --install -d Debian
PS> wsl --shutdown
```

`-l -v`: qué hay instalado y en qué versión.
`--shutdown`: lo apaga.

Dentro de Ubuntu

```
$ whoami
$ uname -r # kernel de Linux
$ sudo apt update
$ explorer.exe . # abre la carpeta
                  # en Windows
```

`uname -r` muestra el kernel real de WSL 2.
`explorer.exe` desde Linux: la integración de la que hablamos.

Qué comprobar

La terminal de Ubuntu responde, `apt` instala paquetes y los archivos se ven desde ambos lados.

Misma CLI, distinto lugar

Lo que practicas en WSL (permisos, usuarios, servicios, nginx, gcloud) se traslada casi igual a la VM de GCP. Cambia dónde corre: WSL en tu PC, la VM en la nube.

La idea que se repite

Servidor web y servidor de aplicaciones, hipervisor y máquina virtual, WSL y VM: en todos los casos, cada capa tiene una responsabilidad y se apoya en la de abajo.

El esquema completo

```
Tu PC (Windows)
  WSL 2 -> Ubuntu
Nube (GCP)
  Hipervisor tipo 1 -> VM Ubuntu
```

- ▶ ¿Qué devuelve un servidor web y qué devuelve un servidor de aplicaciones?
- ▶ En el diagrama de bloques, ¿por qué el servidor de aplicaciones no se expone al exterior?
- ▶ ¿En qué se parecen y en qué se diferencian WSL 2 y una máquina virtual de VirtualBox?
- ▶ ¿Por qué WSL 1 sí es distinto de virtualizar con un hipervisor?
- ▶ ¿Qué comando instala WSL y qué instala por defecto?



gracias



Universidad
Tecnológica
del Perú

- ▶ Documentación de NGINX: *Serving Static Content* y *Reverse Proxy*, docs.nginx.com/nginx/admin-guide.
- ▶ Documentación de Python: módulo `http.server` y nota de "no recomendado para producción", docs.python.org/3/library/http.server.html.
- ▶ Documentación de Gunicorn: *Deploying Gunicorn* (Gunicorn detrás de un proxy como nginx), docs.gunicorn.org.
- ▶ Microsoft Learn: *What is the Windows Subsystem for Linux?*, learn.microsoft.com/windows/wsl/about.
- ▶ Microsoft Learn: *Comparing WSL 1 and WSL 2*, learn.microsoft.com/windows/wsl/compare-versions.
- ▶ Microsoft Learn: *Install Linux on Windows with WSL y Basic commands for WSL*, [learn.microsoft.com/windows/wsl/install y /basic-commands](https://learn.microsoft.com/windows/wsl/install%20y/basic-commands).
- ▶ Microsoft Learn: *Windows Subsystem for Linux Architecture* (WSL 2 y la máquina virtual ligera con Hyper-V).
- ▶ Material base: Sesiones 02 y 09 de este curso; identidad visual UTP y estructura docente reutilizadas sin cambios.